

BHTTP/1 — HTTP, in binary

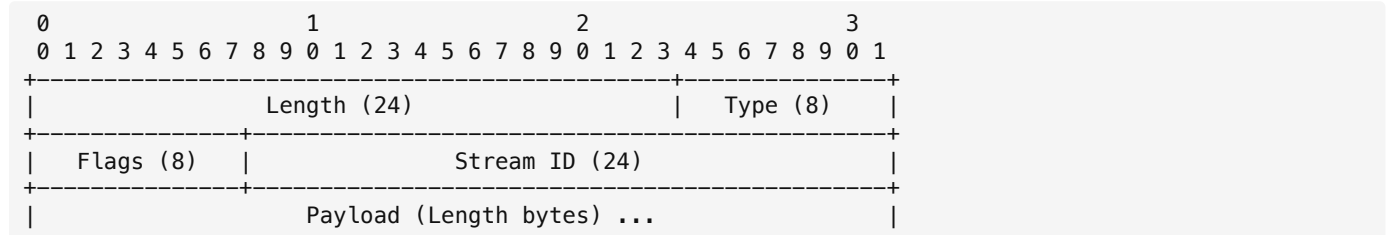
Protocol specification · version 1

The key words MUST, MUST NOT, SHOULD and MAY are used as in RFC 2119. All integers are unsigned and big-endian (network byte order).

1. Model

BHTTP/1 carries HTTP request/response semantics over one TCP connection (default port **9000**) as a sequence of binary **frames**. There is no connection preface. The client sends a request, reads the complete response, and MAY then send another request **on the same connection**. A client MUST NOT open a second connection to make further requests to the same server. The server keeps the connection open until the client closes it (or after 30 s idle, which is implementation-defined).

2. Frame header (8 bytes, fixed)



- **Length**: payload size in bytes, excluding these 8 header bytes (0 ... 16 777 215).
- **Type**: 0x00 DATA, 0x01 HEADERS. Every other value is **unknown** (see §3).
- **Flags**: 0x01 END_STREAM, set on the last frame of a request or response. Senders MUST set every other bit to 0; receivers MUST ignore bits they do not know.
- **Stream ID**: ties a response to its request. Requests use 1, 2, 3, ... (strictly increasing). 0 is reserved for connection-level frames.

3. Unknown frames (the rule that leaves room for version 2)

A receiver that reads a frame whose Type it does not know **MUST skip it cleanly**. It reads and discards exactly Length payload bytes and then carries on with the next frame. It MUST NOT answer with an error, close the connection, or change any request state. The rule applies on every stream, stream 0 included, and also between the frames of a request or response. Because every frame states its own length, a v1 peer can always resynchronise past features it has never heard of.

4. Messages

A **request** is one HEADERS frame followed by zero or more DATA frames. A **response** is the same thing, on the same Stream ID. END_STREAM marks the last frame. A message with no body is a single HEADERS frame with END_STREAM set. The whole header block MUST fit in one HEADERS frame, and receivers MUST accept header blocks up to 65 536 bytes. DATA payloads are opaque bytes. Senders SHOULD keep each DATA frame at 16 384 bytes or fewer, and receivers MUST accept any length. END_STREAM, not content-length, decides where a body ends.

5. Header block encoding

The payload of a HEADERS frame is a list of fields with no count and no terminator; the frame Length bounds it.

```
field = tag:1 [ string ] string ; name string present only when tag = 0
string = len bytes
len = 0xxxxxxx ; 0 ... 127, one byte
      | 1xxxxxxx xxxxxxxx ; 0 ... 32 767, two bytes (15 bits)
```

tag = 0 means a literal name follows. **tag 1–10** names a field from the static table below. These are the ten names BHTTP/1 actually sends. Tags 11–255 are reserved; a v1 receiver treats them as malformed.

#	name	#	name	#	name	#	name	#	name
1	:method	3	:status	5	user-agent	7	server	9	content-type
2	:path	4	host	6	accept	8	date	10	content-length

Names are lowercase visible ASCII (0x21–0x7E). Pseudo-header names begin with :, MUST be sent through the table and MUST NOT appear as literals. Values MUST NOT contain NUL, CR or LF. A request carries exactly one :method and one :path, and the :path begins with /. A response carries exactly one :status as three ASCII digits.

6. Server behaviour

The server percent-decodes :path, drops any ?query/#fragment and maps the result under its document root. A path ending in /, or naming a directory, maps to index.html. It answers 200 with the file as DATA frames, plus content-type, content-length, server and date. HEAD gets the same headers with no body. Errors carry a short text/plain body.

status	when
400	malformed request: bad header block, reserved tag, missing or duplicate pseudo-headers, header block over 64 KiB
403	path contains a . . segment, or resolves (via symlinks) outside the root
404	no regular file at that path
405	method other than GET / HEAD

Stream errors (everything above) get a response, and **the connection stays open**. **Connection errors** are cases where the frame sequence itself can no longer be trusted: a HEADERS frame whose Stream ID is 0 or not greater than the previous one, DATA outside an open request, or a frame cut short by EOF. For these the server answers 400 on that Stream ID if it still can, and then closes the connection. A request body, if one is sent, is read and discarded.

7. Client behaviour (curl)

The client sends one request at a time and reads the full response before sending the next. It writes response bodies to stdout. With -v it hexdumps every frame it sends (>) and receives (<) to stderr. Exit status: 0 if every response was 1xx-3xx, 4 if any was 4xx, 5 if any was 5xx, 3 on a protocol error, 2 if it could not connect, 1 on bad usage.

8. Why these widths (and why HTTP/2 chose 24 / 8 / 8 / 31)

HTTP/2 multiplexes many concurrent streams, so it needs a large stream space. Client and server both open streams, split between odd and even IDs, which costs a bit. The top bit is reserved so the ID still fits a signed 32-bit integer, as in Java. That gives 31 bits. Its 24-bit length allows frames up to 16 MiB but defaults to 16 KiB, and the header stays at 9 bytes. One type byte and one flags byte cover all of its frame types and their per-frame booleans.

BHTTP/1 keeps 24 / 8 / 8 and shrinks the Stream ID to 24 bits, which makes the header exactly **8 bytes**: one aligned 64-bit word.

- **Length 24**: a 16-bit length would cap frames at 64 KiB. 24 bits costs one byte and matches HTTP/2. Receivers stream DATA instead of buffering whole frames, so a large Length costs no memory, and header blocks have their own 64 KiB cap.
- **Type 8**: v1 uses 2 of 256 values. Together with §3 that leaves 254 types for later versions.
- **Flags 8**: v1 uses 1 bit. The other 7 are free for later versions, and receivers ignore them for now.
- **Stream 24**: v1 does not multiplex, so the ID only correlates a response with its request. That allows 16.7 M requests per connection, more than any keep-alive will use. Because the field already exists, a v2 can add pipelining or multiplexing without changing the header.

Headers. Two of HPACK's mechanisms: an index for the names we actually send, and length-prefixed literals for everything else. HPACK's dynamic table and Huffman coding are left out. A typical request is 57 bytes on the wire (8-byte header plus a 49-byte block). The same request in HTTP/1.1 text is 85 bytes.

Evolving. A v2 that wants new frame types or table entries first sends a new frame type, e.g. SETTINGS. A v1 peer skips it (§3) and never acknowledges it, so both sides stay on v1 and nothing breaks.